Entregável 1 explicado

Rede de distribuição: histórico de períodos

Algoritmos e Estruturas de Dados – BCC – 2026.2

O que este documento cobre

- Como o arquivo `e1c.py` funciona, parte por parte, com os trechos de código.
- Os conceitos por trás de cada parte: tabela hash, manutenção incremental, programação dinâmica (Kadane), pilha e análise amortizada.
- Um exemplo simulado à mão, a tabela de complexidades e as perguntas prováveis da entrevista.

Sumário

- 1. O problema do Entregável 1
- 2. Visão geral do programa
- 3. Leitura da entrada
- 4. Registro dos depósitos e o total incremental
- 5. Períodos e lotes
- 6. MELHOR: soma máxima consecutiva (Kadane online)
- 7. PROSPERIDADE: a pilha de prosperidade
- 8. Simulação completa do exemplo
- 9. Complexidades
- 10. Conceitos teóricos para revisar
- 11. Perguntas prováveis da entrevista
- 12. Como o código foi testado

1. O problema do Entregável 1

A rede tem depósitos, cada um com um identificador e um lucro (que pode ser negativo). O tempo é dividido em **períodos** numerados a partir de 1. O período 1 é o estado logo após o *setup*; em cada período k > 1 chega um **lote** de operações e, ao fim dele, calcula-se o lucro do período:

$\mathbf{L_k}$ = soma dos lucros de todos os depósitos vivos ao fim do lote k.

No E1 aparecem só quatro operações:

Operação	O que faz	Imprime?
INSERIR id x y L r	Cria um depósito com lucro L. Vem seguida de r linhas de rotas (que não contam em q).	Não
LUCRO id L	Substitui (não soma) o lucro do depósito id por L.	Não
MELHOR	Intervalo [i, j] de períodos consecutivos com a maior soma de L, entre os já registrados.	i j S
PROSPERIDADE	Estado da pilha de prosperidade após o período anterior.	0 ou h b s i r

O enunciado exige eficiência: o total do período **não pode** ser obtido varrendo todos os depósitos, e MELHOR **não pode** reexaminar todo o histórico. A meta é O(1) (esperado ou amortizado) por operação.

2. Visão geral do programa

O arquivo é um único programa: o ponto de ancoragem `main()` lê a entrada, mantém o estado e imprime as respostas. As partes do histórico ficam em funções pequenas:

Parte	Função / variável	Papel
Registro	lucros (dict), total (int)	Lucro de cada depósito e a soma de todos
Kadane online	novo_kadane, atualizar_kadane, melhor	Responde MELHOR em O(1)
Pilha	pilha (list), soma_pilha, atualizar_pilha, prosperidade	Responde PROSPERIDADE em O(1)
Saída	saida (list de strings)	Acumula as respostas e imprime tudo no final

O fluxo é sempre o mesmo:

```
ler setup  ->  registrar período 1
para cada lote k = 2 .. P+1:
    ler q
    executar as q operações (MELHOR/PROSPERIDADE enxergam até k-1)
    registrar período k  (atualiza Kadane e pilha com L_k = total)
imprimir as respostas
```

3. Leitura da entrada

A entrada inteira é lida de uma vez e quebrada em **tokens** (palavras). Um índice `pos` aponta para o próximo token ainda não lido. Isso torna o programa independente das quebras de linha e é rápido em Python (uma única chamada de leitura).

```
tokens = sys.stdin.read().split()
n, m, T = int(tokens[0]), int(tokens[1]), int(tokens[2])
pos = 3
```

Depois do cabeçalho vêm n depósitos (4 inteiros cada) e m rotas (3 inteiros cada). No E1 as rotas não são usadas, mas **precisam ser puladas**: basta avançar `pos` em 3·m.

```
for _ in range(n):
    ident, x, y, lucro = (int(t) for t in tokens[pos:pos + 4])
    lucros[ident] = lucro
    total += lucro
    pos += 4
pos += 3 * m                    # rotas: lidas e descartadas no E1
```

Quantos tokens cada operação consome depois do seu nome:

Operação	Tokens consumidos	Detalhe
INSERIR	5 + 3·r	id x y L r e depois as r rotas u v c
LUCRO	2	id L
MELHOR, PROSPERIDADE	0	Não têm argumentos
DIST (não aparece no E1)	1	Tratada só por segurança

Armadilha

- Se o INSERIR pular só 5 tokens, as rotas são lidas como se fossem operações e todo o resto da entrada sai desalinhado. O enunciado avisa: uma leitura errada invalida todos os testes do estágio.
- O valor T é lido, mas não é usado no E1: só a operação REMOVER (do E2) fecha depósitos.

4. Registro dos depósitos e o total incremental

4.1 Por que um dicionário (tabela hash)?

Precisamos achar o lucro de um depósito pelo seu id. As alternativas:

Estrutura	Busca por id	Problema
Vetor indexado por id	O(1)	ids vão até 10^9: seriam bilhões de posições
Lista de pares (id, lucro)	O(n)	Cada LUCRO varreria até 5000 depósitos
Árvore balanceada	O(log n)	Mantém ordem, que não precisamos
Tabela hash (dict)	**O(1) esperado**	Nenhum: é a escolha

O `dict` do Python é uma tabela hash: a chave passa por uma função de hash que indica onde guardar o valor. Com poucas colisões, inserir, buscar e atualizar custam O(1) **em média (esperado)**. No pior caso (todas as chaves colidindo) seria O(n), por isso dizemos "esperado" e não "pior caso".

4.2 O total incremental

Em vez de somar todos os lucros ao fim de cada lote, a variável `total` é ajustada a cada mudança:

```
# INSERIR
lucros[ident] = lucro
total += lucro

# LUCRO
total += lucro - lucros[ident]  # ajusta pela diferença...
lucros[ident] = lucro           # ...e só depois sobrescreve
```

Invariante: após cada operação, `total == sum(lucros.values())`. Por isso L_k é lido em O(1) ao fim do lote.

> **Armadilha**
> - No LUCRO, a ordem importa: se o valor novo for gravado antes, `lucros[ident]` já é o novo e a diferença dá 0.
> - LUCRO substitui o valor, não soma. Por isso se ajusta pela diferença (novo − antigo).

5. Períodos e lotes

```
k = 1                           # período 1: logo após o setup
atualizar_kadane(kadane, k, total)
soma_pilha = atualizar_pilha(pilha, soma_pilha, k, total)

P = int(tokens[pos]); pos += 1
for _ in range(P):              # um lote por período (2 até P+1)
    q = int(tokens[pos]); pos += 1
    for _ in range(q):
        ...                     # executa a operação
    k += 1                      # fim do lote: registra L_k
    atualizar_kadane(kadane, k, total)
    soma_pilha = atualizar_pilha(pilha, soma_pilha, k, total)
```

O período só é registrado **depois** do lote. Isso resolve sozinho uma regra do enunciado: MELHOR e PROSPERIDADE executadas no lote k devem enxergar o histórico só até k − 1, porque L_k ainda não existe enquanto o lote roda.

- Um lote com q = 0 ainda é um período: o registro fica fora do laço do q, então acontece mesmo assim.
- Cada período é registrado exatamente uma vez (uma chamada de cada função de atualização).

6. MELHOR: soma máxima consecutiva (Kadane online)

6.1 O problema

Dada a sequência L_1, ..., L_{k-1}, achar o intervalo não vazio [i, j] de maior soma. Empate: menor i; persistindo, menor j. Testar todos os intervalos custaria $O(P^2)$ por consulta; recalcular com Kadane a cada consulta custaria O(P). Queremos O(1).

6.2 A ideia: programação dinâmica

Seja **E(k)** a maior soma de um intervalo que **termina exatamente em k**. Ou ele é só o período k, ou é o melhor que terminava em k − 1 estendido com L_k:

$E(k) = L_k + \max(E(k-1), 0)$

Se E(k−1) é negativo, carregá-lo só piora a soma: melhor recomeçar em k. A resposta global é o maior E(k) já visto. Como cada passo só depende do anterior, basta guardar **dois estados**, atualizados quando um período é registrado. Isso é o "Kadane online".

Campo	Significado
fim_soma, fim_ini	Soma e início do melhor intervalo que termina no último período (E(k))
mi, mj, ms	Melhor intervalo global [mi, mj] e sua soma

```
def atualizar_kadane(kadane, k, L):
    if k > 1 and kadane["fim_soma"] >= 0:
        kadane["fim_soma"] += L          # estende
    else:
        kadane["fim_soma"] = L           # recomeça em k
        kadane["fim_ini"] = k

    s, i = kadane["fim_soma"], kadane["fim_ini"]
    if (k == 1 or s > kadane["ms"]
            or (s == kadane["ms"] and (i, k) < (kadane["mi"], kadane["mj"]))):
        kadane["mi"], kadane["mj"], kadane["ms"] = i, k, s
```

6.3 Os desempates

- **Por que >= 0 e não > 0?** Se a soma anterior é exatamente 0, estender ou recomeçar dá a mesma soma; estendendo, o início fica mais antigo (menor i), que é o que o enunciado pede.
- **Comparação com o global:** troca se a soma for estritamente maior, ou igual com (i, j) lexicograficamente menor. Em Python, a comparação de tuplas `(i, k) < (mi, mj)` faz exatamente isso.
- **Sempre não vazio:** com todos os L negativos, o resultado é o maior elemento isolado (o primeiro, se houver empate), porque o estado sempre recomeça em um intervalo de um período.
- **k == 1:** o primeiro período inicializa o global, já que ainda não há nada para comparar.

A consulta só lê o estado:

```
def melhor(kadane):
    return f"{kadane['mi']} {kadane['mj']} {kadane['ms']}"
```

7. PROSPERIDADE: a pilha de prosperidade

7.1 As regras

Cada elemento é um par **(j, r)**: j é um período com lucro positivo e r > 0 é a parte desse lucro que ainda não foi "comida" por prejuízos posteriores. A base é o ganho mais antigo; o topo, o mais recente. Após calcular L_k:

- **$L_k > 0$:** empilha (k, L_k).
- **$L_k = 0$:** nada muda.
- **$L_k < 0$:** o prejuízo d = $-L_k$ consome a pilha a partir do topo. Se o topo tem r > d, ele vira (j, r − d) e acabou. Se r <= d, ele é desempilhado e d diminui r. Se a pilha esvazia e sobra d, a sobra é descartada.

```
def atualizar_pilha(pilha, soma, k, L):
    if L > 0:
        pilha.append([k, L])
        soma += L
    elif L < 0:
        d = -L
        while d > 0 and pilha:
            j, r = pilha[-1]
            if r > d:                 # topo absorve o prejuízo todo
                pilha[-1][1] = r - d  # mantém o j original
                soma -= d
                d = 0
            else:                     # r <= d: topo consumido (inclusive exato)
                pilha.pop()
                soma -= r
                d -= r
    return soma                       # L == 0: nada muda
```

7.2 Detalhes que importam

- **r > d, não r >= d:** com r = d o topo deve ser desempilhado; com >= ficaria um resíduo 0, e os resíduos precisam ser estritamente positivos.
- **O j não muda** quando o resíduo é reduzido: ele registra quando o ganho aconteceu.
- **soma_pilha** é mantida em cada push, pop e redução. Assim PROSPERIDADE não percorre a pilha.
- Os elementos são listas `[j, r]` (e não tuplas) para que o resíduo do topo possa ser alterado no lugar.
- **Invariante:** se a pilha não está vazia e b é o j da base, soma_pilha = L_b + ... + L_k. O intervalo [b, k] é o período de prosperidade corrente.

```
def prosperidade(pilha, soma):
    if not pilha:
        return "0"
    b = pilha[0][0]                   # período da base
    i, r = pilha[-1]                  # topo
    return f"{len(pilha)} {b} {soma} {i} {r}"
```

7.3 Por que é O(1) amortizado se existe um while?

Um único prejuízo grande pode desempilhar muitos elementos de uma vez, então uma chamada isolada pode custar O(h). Mas olhando a sequência inteira de períodos:

- **Método da contagem (agregado):** cada elemento é empilhado uma vez e desempilhado no máximo uma vez. Em P períodos há no máximo P pushes, logo no máximo P pops no total. O custo total é O(P), ou seja, O(1) por período em média.
- **Método do potencial:** defina Φ = altura da pilha. Um push custa 1 e aumenta Φ em 1 (custo amortizado 2). Desempilhar t elementos custa t + 1 e diminui Φ em t (custo amortizado 1). Todo período custa O(1) amortizado.

> **Frase pronta para a entrevista**
> • "Cada elemento entra na pilha uma vez e sai no máximo uma vez; então o total de iterações do while em toda a execução é limitado pelo número de pushes, que é no máximo P. Por isso o custo é O(1) amortizado por período, mesmo com o laço."

8. Simulação completa do exemplo

Lucros dos períodos no exemplo do enunciado: L = 22, −8, 15, −29, 11.

k	L_k	Kadane: E(k), início	Melhor global	Pilha após o período	soma
1	22	22, início 1	1 1 22	[(1,22)]	22
2	−8	14, início 1 (14 >= 0: estende)	1 1 22	[(1,14)]	14
3	15	29, início 1	1 3 29	[(1,14), (3,15)]	29
4	−29	0, início 1	1 3 29	[ ] (15 e depois 14 consumidos exatamente)	0
5	11	11, início 1 (0 >= 0: estende)	1 3 29	[(5,11)]	11

Período 4 em detalhe: d = 29. Topo (3, 15): 15 <= 29, desempilha, d = 14. Novo topo (1, 14): 14 <= 14, desempilha, d = 0. A pilha fica vazia sem sobra.

Respostas das consultas (cada lote enxerga até o período anterior):

Lote	Consulta	Saída	Por quê
2	PROSPERIDADE	1 1 22 1 22	Pilha após o período 1: [(1,22)]
2	MELHOR	1 1 22	Só existe o período 1
3	PROSPERIDADE	1 1 14 1 14	Pilha [(1,14)]
3	MELHOR	1 1 22	22 > 22 − 8
4	PROSPERIDADE	2 1 29 3 15	Altura 2, base 1, soma 29, topo (3,15)
5	PROSPERIDADE	0	Pilha vazia após o período 4
5	MELHOR	1 3 29	22 − 8 + 15 = 29
6	PROSPERIDADE	1 5 11 5 11	Pilha [(5,11)]
6	MELHOR	1 3 29	[5,5] soma 11 < 29: o melhor continua histórico

9. Complexidades

n = depósitos vivos, P = períodos registrados, h = altura da pilha.

Operação	Tempo	Justificativa
Leitura do setup	O(n + m)	4 tokens por depósito; rotas só puladas
INSERIR	O(1) esperado	Uma inserção no dict e uma soma (as rotas são só puladas: O(r))
LUCRO	O(1) esperado	Uma busca e uma atribuição no dict
Registrar período	O(1) amortizado	Kadane é constante; pilha: cada elemento sai no máximo uma vez

Operação	Tempo	Justificativa
MELHOR	O(1)	Lê o estado mantido
PROSPERIDADE	O(1)	len, base e topo por índice; soma já mantida
Programa inteiro	O(tamanho da entrada)	Cada token é processado um número constante de vezes

Espaço: O(n) para o dicionário, O(P) no pior caso para a pilha, O(1) para o Kadane, além da entrada e da saída.

10. Conceitos teóricos para revisar

Pior caso, esperado e amortizado

Tipo	O que garante	Exemplo no código
Pior caso	Limite para **toda** execução da operação, sem exceção	MELHOR e PROSPERIDADE: O(1)
Esperado	Média sobre a aleatoriedade (ex.: distribuição das chaves no hash); uma operação pode ser lenta com azar	Busca no dict: O(1) esperado
Amortizado	Média sobre uma **sequência** de operações, sem probabilidade; algumas são caras, mas o total é limitado	Pilha: O(1) amortizado; append em list também

Tabela hash

- Uma função de hash leva a chave a uma posição de um vetor interno.
- Colisões (chaves diferentes na mesma posição) são resolvidas por encadeamento ou endereçamento aberto (o Python usa endereçamento aberto).
- Quando a tabela enche, ela é redimensionada (realocação): isso é raro e amortizado.
- Daí O(1) esperado e amortizado, mas O(n) no pior caso.

Programação dinâmica

- Resolve um problema combinando respostas de subproblemas menores, sem recalculá-los.
- No Kadane, o subproblema é "melhor intervalo que termina em k", e a recorrência usa só o valor de k − 1.
- "Online" quer dizer que a resposta é atualizada a cada novo dado, sem guardar nem reler o histórico.

Pilha

- Estrutura LIFO (último a entrar, primeiro a sair): push e pop no topo.
- Em Python, uma list serve de pilha: append e pop no fim custam O(1) amortizado.
- A pilha de prosperidade é LIFO porque o prejuízo consome primeiro o ganho mais recente.

11. Perguntas prováveis da entrevista

Pergunta	Ideia central da resposta
Por que dicionário e não vetor ou lista?	ids até 10^9 impedem vetor; lista é O(n) por busca; hash dá O(1) esperado; árvore seria O(log n) sem precisar de ordem.
Como o total é mantido sem varrer?	Ajuste incremental: += L no INSERIR, += (novo − antigo) no LUCRO. Invariante: total = soma dos lucros.
Por que a pilha é O(1) amortizado se há um while?	Cada elemento é empilhado uma vez e desempilhado no máximo uma vez; total de pops <= total de pushes <= P.
Como MELHOR evita reexaminar o histórico?	Kadane online: guarda o melhor que termina no último período e o melhor global; atualiza ao registrar cada período.
Como trata empates no MELHOR?	Estende com soma anterior >= 0 (mantém menor i) e compara (i, j) lexicograficamente no global.
Por que a consulta do lote k vê só até k − 1?	O período k só é registrado depois do lote.
Por que r > d e não r >= d?	Com r = d o topo deve sair; com >= sobraria resíduo 0.
E se mudar o desempate para o maior i?	Estender só com soma anterior > 0 e trocar a comparação do global para preferir i maior.

Treino sugerido

- Pegue uma sequência curta de L (5 ou 6 valores, com zeros e prejuízos exatos) e simule no papel o Kadane e a pilha, como na seção 8.
- Explique, sem olhar o código, a entrada, a saída, o invariante e a complexidade de atualizar_kadane e atualizar_pilha.

12. Como o código foi testado

- **Exemplo do enunciado:** uma entrada com os mesmos lucros (22, −8, 15, −29, 11) produziu exatamente as saídas da tabela da seção 8.
- **Força bruta:** uma versão ingênua (testa todos os intervalos e reconstrói a pilha do zero) foi comparada com o e1c.py em 3000 entradas aleatórias pequenas, com lucros negativos, zeros, prejuízos exatos e lotes vazios. Nenhuma diferença.
- **Desempenho:** n = 5000, 25000 rotas, P = 10^5 e 3×10^5 operações rodaram em cerca de 0,5 s.